

Modül 19: Sanal İş Parçacıkları

Sanal iş parçacığı, işletim sisteminin değil JVM'in zamanladığı bir iş parçacığıdır. Aynı `Thread` API'si, aynı kod, aynı hata ayıklayıcı. Değişen şey fiyatı.

Bir platform iş parçacığı yaklaşık bir megabayt yığın alanı ve bir işletim sistemi zamanlama yuvası ayırır. Sanal iş parçacığı ise gerektikçe büyüyen bir yığın nesnesidir. Bu tek değişiklik bloke olmayı yeniden ucuzlatır, ve bu modül çoğunlukla bundan ne çıktığıyla ilgili.

Diğer kaynaklar hakkında bir uyarı: sanal iş parçacığı sabitlenmesi hakkında yazılmış materyalin çoğu güncelliğini yitirmiştir. Bu modül davranışı tekrarlamak yerine ölçüyor.

Neler Öğreneceksiniz

- Binme, inme, ve taşıyıcı iş parçacığının ne olduğu
- Savı kuran kıyaslama, kendi makinenizde ölçülmüş
- Bugün neyin sabitlediği, ki eskiye göre çok daha azı
- `ScopedValue`, ve `ThreadLocal` neden bir milyon iş parçacığına ölçeklenmiyor
- İnsanların onları yanlış kullandığı dört yol, üçü eski alışkanlıklardan

Başka Bir Dilden Geliyorsanız

	Go	Python	Java
Hafif eşzamanlılık	goroutine	<code>asyncio</code> görevi	sanal iş parçacığı
Bloke eden çağrı	sorun değil, zamanlayıcı halleder	döngüyü bloke eder	sorun değil, iş parçacığı iner
Renk problemi	yok	<code>async</code> / <code>await</code> API'nizi ikiye böler	yok
Birim maliyet	~2KB	küçük	küçük, gerektikçe büyür

Sanal iş parçacıkları Java'yı Go'nun başından beri olduğu yere getiriyor: sıradan, bloke eden, ardışık kod yazıyorsunuz ve çalışma zamanı onu ölçeklendiriyor. Kütüphaneyi ikiye bölen bir `async` anahtar kelimesi yok.

Ders

Binme ve taşıyıcılar

```
Thread.ofVirtual().name("worker").start(() -> ...);
```

İş parçacığını yazdırmak `VirtualThread[#33,worker]/runnable@ForkJoinPool-1-worker-1` gibi bir şey gösterir. O `ForkJoinPool-1-worker-1` **taşıyıcıdır**: sanal iş parçacığının şu an üzerine bindiği gerçek bir platform iş parçacığı.

Sanal iş parçacığı bloke olduğunda iner ve taşıyıcı bir başkasını alır. Tüm mekanizma bu. Bloke olmak artık bir iş parçacığına mal olmuyor.

Gerçekten kullanacağınız executor bir havuz değil, **görev başına bir iş parçacığı** oluşturur:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ... }
```

Boyutlandırılacak bir şey yok, çünkü kıt kaynak iş parçacıkları değil. `VirtualThreads.java` dosyasından:

```
100000 virtual threads, each sleeping 10ms: 216ms
```

Kıyaslama

`Benchmark.java` dosyasından, 8 çekirdekli bir makinede her biri 100ms bloke olan 5.000 görev:

virtual thread per task	127ms
platform pool of 200	2716ms
platform pool of 50	10750ms

Aritmetik basit. 200'lük bir havuzda aynı anda en fazla 200 görev bloke olabilir, dolayısıyla 5.000 görev 25 grup halinde 100ms sürer. 50'lik havuz 100 grup alır. Sanal iş parçacıklarının böyle bir sınırı yoktur, dolayısıyla 5.000'i birden bloke olur.

Bunun ortadan kaldırdığı seçim. IO için bir platform havuzunu boyutlandırmak her zaman kötü bir takastı: çok az iş parçacığı ve istekler bloke olanların arkasında sıraya girer, çok fazla ve her biri için bir megabayt yığın öderken işletim sistemi zamanlayıcısı çırpınmaya başlar.

Alışılmış kaçış eşzamansız koddu, ki çalışır ve size okunabilir yığın izlerini, adım adım hata ayıklamayı ve sıradan `try / catch` kullanımını kaybettirir. Sanal iş parçacıkları eşzamansız ölçeklenebilirliği düz bloke eden koda geri veriyor.

Yardımcı olmadıkları yer: CPU ağırlıklı iş. Hiç bloke olmayan bir görev hiç inmez, ve çekirdek sayınızdan fazla işi paralel çalıştıramazsınız. Orada sabit bir platform havuzu hâlâ doğrudur.

Sabitlenme, bugün

Sabitlenme, sanal iş parçacığının inememesi ve dolayısıyla taşıyıcısının bloke kalıp bir platform iş parçacığının boşa harcanmasıdır.

Java 21 ile 23 arasında, `synchronized` bloğu içinde bloke olmak taşıyıcıyı sabitliyordu, ve o dönemden kalma her yazı `synchronized` yerine `ReentrantLock` kullanmanızı söyler.

JDK 24'te gelen JEP 491 bunu kaldırdı. `PinningToday.java` dosyasından, her biri kendi kilidini tutarken 100ms uyuyan 2.000 görev:

```
cores, and therefore carriers: 8
elapsed: 131ms

If synchronized still pinned, only 8 tasks could sleep at
once, so this would take about 25000ms.
```

Hâlâ sabitleyenler:

1. **Yerel bir metot ya da yabancı fonksiyon çağırısı.** JVM, denetlemediği bir yığını indiremez.
2. **Bloke olan bir sınıf ilklendiricisi.** Nadir, ve genellikle zaten bir tasarım problemi.

İkisi de eski kuraldan çok daha dar.

Sabitlenme ile çekişme farklı şeylerdir. Kilit hâlâ kilittir: bin sanal iş parçacığı tek bir monitör için yarışırsa 999'u bekler. Sabitlenme bir *taşıyıcıyı* harcar; çekişme *zaman* harcar. JEP 491 birincisini çözdü ve ikincisine dokunamazdı, çünkü sıraya sokmak kilidin işidir.

Kendi kodunuzu, bu yazı dahil hiçbir makaleye güvenmek yerine `-Djdk.tracePinnedThreads=full` ile denetleyin.

ScopedValue

JDK 25'te kesinleşti. Yaygın durumda `ThreadLocal` yerine geçer.

Problem: bir `ThreadLocal`, birinin `remove()` çağırmaı hatırlamasına kadar yaşayan, değiştirilebilir, sınırsız, iş parçacığı başına bir girdidir. Birkaç yüz havuzlanmış iş parçacığıyla bu katlanılabildi. Bir milyon sanal iş parçacığıyla değil.

```
private static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, "ada").run(() -> {
    handleRequest();           // CURRENT_USER.get() works at any depth
});
// binding is gone here, including if the body threw
```

Unutulacak bir `remove()` yok. Bağlama tam olarak `run()` süresince yaşar.

Değişmezdir, ki asıl nokta bu. Bir `ThreadLocal`, ona ulaşabilen her şey tarafından, her derinlikte ayarlanabilir ve değişiklik iş parçacığının ömrü boyunca kalır. Bir `ScopedValue` yalnızca iç içe bir kapsamla gölgelenebilir, ki o da kodda görünür.

Bilinmeye değer bir kısıt:

```
child thread sees it bound? false
```

Düz bir çocuk iş parçacığı bağlamayı **miras almaz**, bilerek: ebeveynin kapsamı çocuk hâlâ çalışırken bitebilir. Miras alma yapılandırılmış eşzamanlılık gerektirir, ki o da JEP 533 ve bu sürümde hâlâ preview. Modül 22 onu anlatıyor.

Kullanım	Ne için
<code>ScopedValue</code>	bir çağrı süresince salt okunur değer: mevcut kullanıcı, istek kimliği
<code>ThreadLocal</code>	gerçekten değiştirilebilir iş parçacığı başına durum: tekrar kullanılan bir tampon

Dört yanlış kullanım

`CommonMisuses.java` dosyasından:

- Onları havuzlamak.** `newFixedThreadPool(200, Thread.ofVirtual().factory())` sizi 200 ile sınırlar ve hiçbir şey kazandırmaz. Havuzlar pahalı bir kaynağı tekrar kullanır; bunlar pahalı değil.
- Havuzun aynı zamanda hız sınırınız olduğunu unutmak.** Bir platform havuzu iki iş yapıyordu: iş parçacığı sağlamak ve eşzamanlılığı sınırlamak. Sanal iş parçacıkları yalnızca birincisinin yerine geçer. 10'luk bir havuz veritabanınızı koruyorsa, onu kaldırmak korumayı kaldırır ve bunu sabah üçte öğrenirsiniz. Kısıtı ima etmek yerine belirten bir `Semaphore` kullanın.
- CPU ağırlıklı iş.** Aritmetik yapan on bin sanal iş parçacığı çekirdek sayınızı yenmez; yalnızca zamanlama maliyeti ekler.
- Ölçekte `ThreadLocal`.** İş parçacığı başına bir girdi, ve artık bir milyon tane var.

Sanal iş parçacıkları bloke olmayı ucuzlatır. Paylaşmayı güvenli kılmazlar, ve bir CPU çarpanı değildirler. Modül 18'deki yarışlar, görünürlük ve kilitler hakkındaki her şey kelimesi kelimesine geçerlidir.

Çalıştırın

```
java modules/19-virtual-threads/examples/VirtualThreads.java
java modules/19-virtual-threads/examples/Benchmark.java
java modules/19-virtual-threads/examples/PinningToday.java
java modules/19-virtual-threads/examples/ScopedValues.java
java modules/19-virtual-threads/examples/CommonMisuses.java

./scripts/verify-examples.sh modules/19-virtual-threads
```

Kıyaslama yaklaşık on beş saniye sürer, çoğu bilerek platform havuzu durumlarında beklemekle geçer.

Sık Yapılan Hatalar

Sanal iş parçacıklarını havuzlamak. Görev başına bir tane; tekrar kullanılacak bir şey yok.

İstemeden bir hız sınırını kaybetmek. Eşzamanlılığı `Semaphore` ile açıkça sınırlayın.

Onları CPU ağırlıklı iş için kullanmak. Sınır hâlâ çekirdekler.

Ölçekte `ThreadLocal` tutmak. İş parçacığı başına bir girdi, ve artık milyonlarca var.

`synchronized` kullanmaktan kaçınma tavsiyesine uymak. Bu JDK 24 öncesinde doğrudu, şimdi değil. Varsaymak yerine ölçün.

Eşzamanlılık hatalarını düzelttiklerini sanmak. İş parçacığı *maliyetini* düzeltirler. Yarışlar ve görünülük el değmeden durur.

Bir çocuk iş parçacığının `ScopedValue` miras almasını beklemek. Yapılandırılmış eşzamanlılık olmadan almaz.

Özet Çıkarımlar

- **Sanal iş parçacığı bloke olduğunda iner**, taşıyıcısını serbest bırakarak. Tüm fikir bu.
- **Havuz değil, görev başına bir iş parçacığı.**
- **JEP 491, JDK 24'te `synchronized` sabitlenmesini kaldırdı.** Yerel çağrılar hâlâ sabitler.
- **Sabitlenme taşıyıcı harcar; çekişme zaman harcar.** Farklı problemler.
- **`ScopedValue` değişmez ve kapsamla sınırlıdır**, temizlenecek hiçbir şey yok.
- **Bloke olmayı ucuzlatırlar, paylaşmayı güvenli kılmazlar.** Modül 18 tümüyle geçerli kalır.

Ödev

[homework/README.md](#)

Farkı kendiniz ölçün, sonra kazara kaldırdığınız hız sınırını bulun. Referans çözüm [solutions/19-virtual-threads/](#) altında.